TRABAJO PRÁCTICO - UNIDAD 6

Seguimiento, control y cierre de proyectos

Materia: Gestión de Desarrollos de Software
Unidad 6: Monitoreo, control y reportes de gestión
Alumno: Nicolás Viruel
Caso: FintechApp - Sprint 12 (módulo Transferencias entre usuarios)

Caso en una línea

Equipo de 5 personas, sprint de 10 días hábiles. Al día 7 hay 22 de 40 puntos cerrados, el backend está listo y el frontend está bloqueado por una dependencia de UI.

Actividad 1 - Reporte de avance (día 7)

Estado general (semáforo): ROJO

El sprint no está a tiempo. El objetivo es entregar Transferencias entre usuarios y el frontend, que es la parte visible de ese módulo, está bloqueado. Además el avance va por detrás del ritmo lineal y casi un tercio del tiempo se fue en urgencias no planificadas. Con solo 3 días hábiles por delante, el desvío ya no es un riesgo manejable: es un problema crítico de alcance y de cronograma.

Hitos alcanzados

• Módulo backend (Node.js) de transferencias completado a tiempo.
• 22 puntos de historia cerrados sobre 40 comprometidos.

Trabajo en curso

• Frontend en React del flujo de transferencias, con alto code churn en TransferForm.jsx.
• Corrección de 12 bugs reportados por QA sobre 3.000 líneas nuevas.
• Pipeline de CI/CD, cuyo tiempo pasó de 6 a 19 minutos.

Bloqueos y riesgos (blockers)

• Bloqueo: el frontend depende de una definición de UI que todavía no está disponible. Sin esa pantalla no se puede cerrar el objetivo del sprint.
• Riesgo de capacidad: 18 h de 60 h (30 %) se consumieron en urgencias no planificadas.
• Riesgo de calidad e integración: 12 bugs en código nuevo y un pipeline tres veces más lento frenan el cierre de las historias que sí están en curso.
• Riesgo de planificación: se comprometieron 40 puntos contra una velocity histórica de 35.

Próximos pasos

1. Desbloquear la dependencia de UI en las próximas 24 h (definir o congelar los componentes de TransferForm) y, si hace falta, hacer pair programming entre frontend y quien tenga el diseño.
2. Revisar por qué el pipeline pasó de 6 a 19 minutos antes de seguir sumando cambios.
3. Reordenar el backlog de los 3 días restantes: priorizar el flujo de transferencia y los bugs que impiden el Definition of Done, y bajar de alcance lo que no entre en el pronóstico.
4. Informar a los interesados que el cierre de 40 puntos no es realista si el bloqueo sigue.

Actividad 2 - Cálculo de KPIs

1. Densidad de defectos

Cálculo: 12 bugs / 3.000 líneas × 1.000 = 4 defectos cada 1.000 líneas.

Mide la calidad del código nuevo: cuántos errores aparecen por cada mil líneas. En un módulo de transferencias de una billetera, 4 bugs/KLOC con el frontend todavía abierto indica que QA ya está encontrando fallas antes de cerrar la funcionalidad. Conviene no dar por estable el módulo hasta bajar esa densidad en el flujo de transferencia.

2. Tasa de trabajo no planificado

Cálculo: 18 h / 60 h × 100 = 30 %.

Es el porcentaje del tiempo del equipo que se fue en urgencias o errores no previstos, en lugar del desarrollo planificado del sprint. Casi una de cada tres horas no aportó a las historias comprometidas. Eso explica parte del atraso y reduce la capacidad real para los 3 días que quedan.

3. Velocity real vs. histórica

Referencia — Puntos
Velocity histórica (promedio de 3 sprints) — 35
Compromiso del Sprint 12 — 40
Completados al día 7 — 22
Ritmo actual — 22 / 7 ≈ 3,14 pts/día
Proyección al día 10 si se mantiene el ritmo — 22 / 7 × 10 ≈ 31 pts

Lectura: el equipo se comprometió por encima de su historia (40 contra 35). Al día 7 el ritmo real proyecta unos 31 puntos, por debajo de la velocity histórica y unos 9 puntos por debajo del compromiso. El bloqueo del frontend empuja el escenario pesimista todavía más abajo, porque los puntos que dependen de la UI no se van a cerrar al mismo ritmo. Para el cierre del sprint lo esperable es quedar cerca de 31 puntos, no de 40, salvo que el bloqueo se resuelva de inmediato y no aparezcan más urgencias.

4. Cycle Time vs. Lead Time

• Cycle Time: 4,5 días. Es el tiempo de trabajo desde que el equipo empieza a programar hasta que termina la tarea.
• Lead Time: 11 días. Es el tiempo total desde que se pide la funcionalidad hasta que llega a producción.
• Diferencia: 11 − 4,5 = 6,5 días que no son programación activa.

Esa brecha es espera: cola, revisiones, dependencias o aprobaciones. En este sprint coincide con el frontend trabado por la UI y con el 30 % de horas en urgencias. El equipo no es lento programando (4,5 días); la funcionalidad tarda 11 días en salir porque pasa mucho tiempo detenida.

Actividad 3 - Estadísticas de ejecución

Porcentaje de completitud al día 7

Cálculo: 22 / 40 × 100 = 55 %.

Se completó poco más de la mitad del backlog cuando ya transcurrió el 70 % del sprint (día 7 de 10).

Desviación del cronograma (SV)

El apunte pide asumir un ritmo lineal de 4 puntos por día.

• Puntos esperados al día 7: $4 \times 7 = 28$
• Puntos completados: 22
• $SV = 22 - 28 = -6$ puntos

¿Adelantado o atrasado?

Atrasado. El SV negativo indica que se hizo menos esfuerzo del que correspondía a la fecha de corte. Faltan 6 puntos respecto de la línea ideal. En un burndown, la curva real quedaría por encima de la ideal (queda más trabajo del previsto para este día). El atraso se concentra en el frontend: el backend llegó a tiempo, pero el objetivo del sprint no se cumple solo con la API.

Actividad 4 - Dashboard inteligente

Un dashboard con IA no se limita a mostrar lo ya ocurrido: compara el flujo actual contra la línea base del equipo y avisa antes de que el desvío se vuelva crítico.

Alerta 1 - Code churn en TransferForm.jsx

Alerta: "Reescritura alta y sostenida en TransferForm.jsx durante el Sprint 12. El archivo se modifica muy por encima del patrón habitual del equipo."

Justificación: un pico de code churn es una anomalía de detección temprana. Suele señalar requisitos ambiguos o complejidad técnica mal acotada. Acá encaja con el bloqueo por dependencia de UI: el frontend reescribe la misma pantalla porque el diseño no está cerrado. Conviene intervenir ya (aclarar la interfaz o hacer pair programming) y no esperar al día 10.

Alerta 2 - Tiempo anómalo del pipeline de CI/CD

Alerta: "El pipeline pasó de 6 a 19 minutos (más del triple) sin un aumento de volumen que lo explique."

Justificación: un tiempo de integración muy por encima de la línea base, sin justificación clara en la cantidad de código, es la anomalía de CI/CD del apunte. Marca riesgo de infraestructura o de deuda técnica. Si no se revisa, cada commit del formulario de transferencias va a tardar más en validarse y el cycle time de lo que queda del sprint empeora.

Actividad 5 - Cierre y lección aprendida

Dos actividades obligatorias del cierre

1. Validación de los criterios de aceptación. Antes de dar por cerrado el release hay que demostrar que el producto cumple lo pactado. Incluye el Definition of Done interno (código revisado, pruebas y criterios de calidad), las pruebas de aceptación del usuario (UAT) y la firma de conformidad (sign-off). Aporta un cierre objetivo: protege al equipo de cambios de alcance de último momento y al cliente, porque verifica que la billetera resuelve el negocio de las transferencias.

2. Informe final de gestión. Consolida el desempeño real contra la línea base: alcance entregado, cronograma, métricas y deuda técnica o errores conocidos que pasan a mantenimiento. Aporta rendición de cuentas a los interesados y a la dirección, y deja registrado qué se entregó y qué quedó pendiente. En este sprint, el informe debería dejar explícito el SV de −6 puntos, el bloqueo de UI y los 12 bugs si no se cierran antes del release.

Lección aprendida (resumen generativo)

Lección Aprendida - Dependencia de UI en el frontend: en el Sprint 12 el backend de Transferencias (Node.js) se completó a tiempo, pero el frontend (React) quedó bloqueado por una dependencia de interfaz. El alto code churn en TransferForm.jsx y la brecha entre el Lead Time (11 días) y el Cycle Time (4,5 días) muestran espera y retrabajo por un diseño no cerrado antes de comprometer los puntos. Acción recomendada para futuros proyectos: congelar el contrato de UI (componentes o prototipo validado) antes de estimar el sprint y no iniciar el desarrollo de frontend hasta que esa dependencia esté disponible.